@Aman_Views

★ COMPLETE MANUAL TESTING NOTES ★

Everything from Beginner to Advanced.

PAGE 1 : TESTING FUNDAMENTALS & TESTING LANDSCAPE

1. WHAT IS SOFTWARE TESTING ?

→ Software testing is the process of evaluating a software application to find defects and ensure that it meets the specified requirements and works as expected.

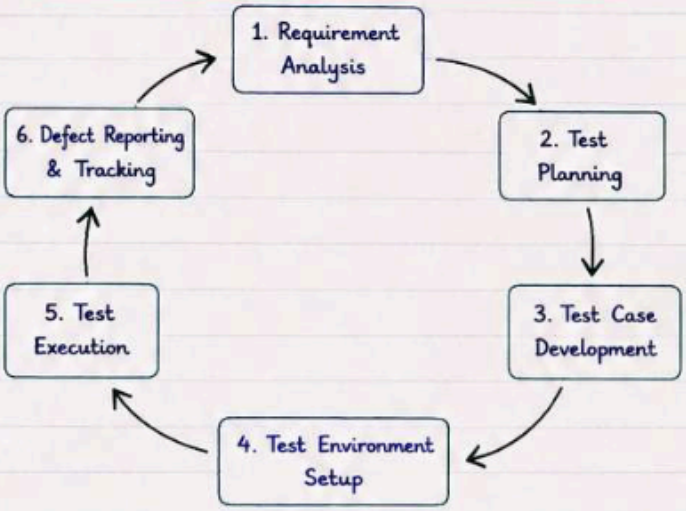
2. WHY SOFTWARE TESTING IS REQUIRED ?

→ To find defects before the product reaches users.
→ To improve quality and reliability.
→ To reduce risks and failures.
→ To save time and cost.
→ To increase customer satisfaction and trust.

3. QA vs QC vs TESTING

Term	Meaning
QA (Quality Assurance)	Process oriented. Focus on preventing defects. Makes sure we build the product right.
QC (Quality Control)	Product oriented. Focus on finding defects. Ensures the product is right.
TESTING	A technical activity to evaluate the product and find defects.

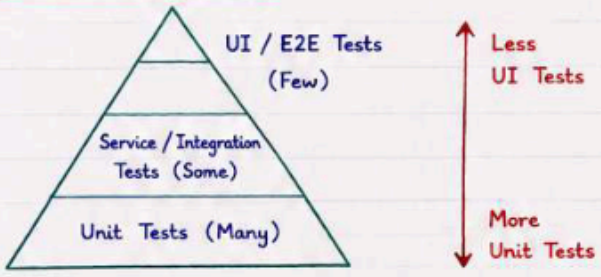
4. BASIC SOFTWARE TESTING LIFE CYCLE



5. ERROR vs DEFECT vs BUG vs FAILURE

- **ERROR** : Human mistake that produces incorrect result.
- **DEFECT** : A flaw in the product (code, design, requirement, docs, etc.).
- **BUG** : A defect in the software that causes unexpected behavior.
- **FAILURE** : When the software does not perform as expected (observed by the user).

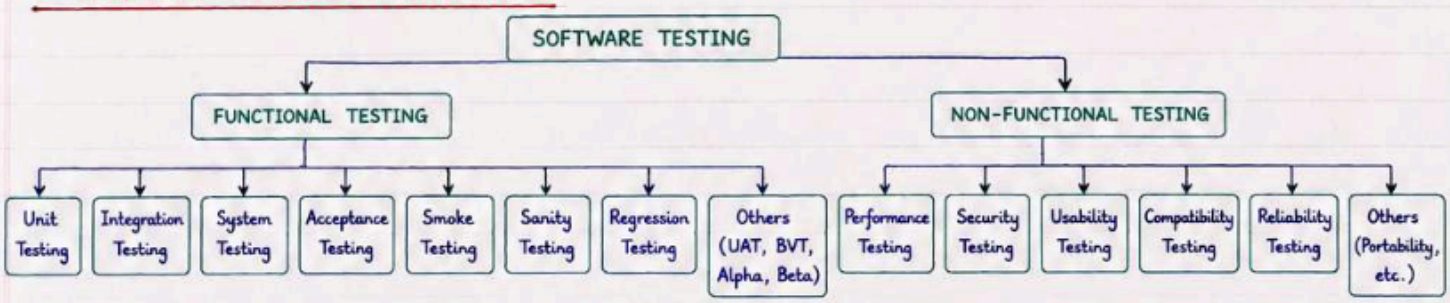
3. TESTING PYRAMID



7. MANUAL TESTING vs AUTOMATION TESTING

Aspect	Manual Testing	Automation Testing
Execution	Done by human	Done by tools / scripts
Cost	Low initial cost	High initial cost
Time	Slow for large test sets	Fast for large test sets
Best For	Exploratory, Ad-hoc, UI	Regression, Repetitive, API
Maintenance	High	High

8. COMPLETE TESTING CLASSIFICATION



@Aman_Views

☆ COMPLETE MANUAL TESTING NOTES ☆

Everything from Beginner to Advanced.

PAGE 2 : SDLC & SOFTWARE DEVELOPMENT MODELS

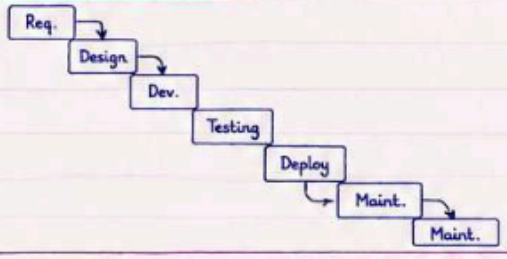
1. SDLC (SOFTWARE DEVELOPMENT LIFE CYCLE)

1. Requirement Gathering → 2. Design → 3. Development (Coding) → 4. Testing → 5. Deployment → 6. Maintenance & Support

☆ Testing starts early and continues throughout the SDLC.

2. SOFTWARE DEVELOPMENT MODELS

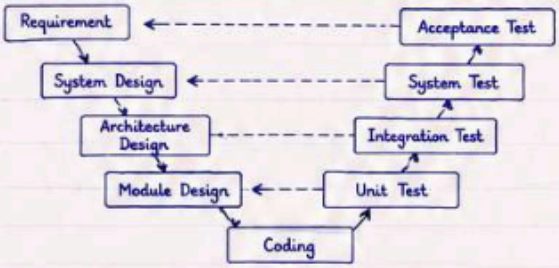
A. WATERFALL MODEL



- Sequential phases, one after another.
- Testing happens after development.
- Suitable for small, stable projects.

\+ Simple to understand and manage.
− Changes are difficult and costly.

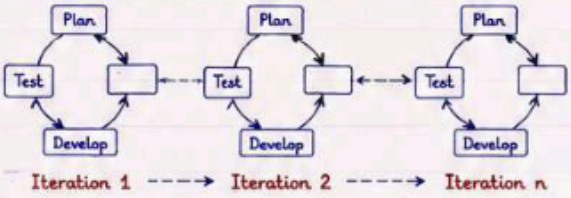
B. V - MODEL (VERIFICATION & VALIDATION MODEL)



- Each development phase has a corresponding testing phase.
- Testing is planned in parallel.
- Suitable for projects with clear requirements.

\+ High quality, less rework.
− Not flexible for changes.

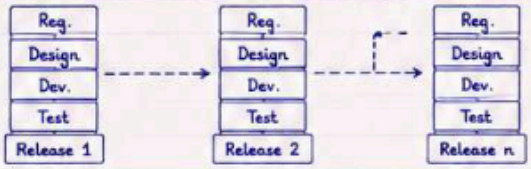
C. ITERATIVE MODEL



- Requirements are divided into small parts (iterations).
- Each iteration goes through all phases.
- Suitable when requirements are partially known.

\+ Early feedback, flexible.
− Needs more planning and tracking.

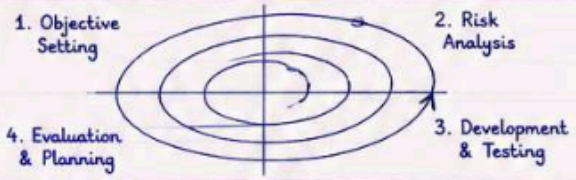
D. INCREMENTAL MODEL



- Product is built in small increments.
- Each increment is tested and released.
- Suitable for medium to large projects.

\+ Early delivery, measurable progress.
− Integration can be complex.

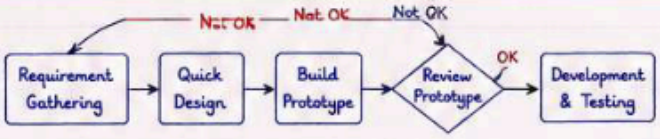
E. SPIRAL MODEL



- Risk-driven model.
- Each loop is a cycle of the project.
- Suitable for large, complex projects.

\+ Handles risks well.
− Costly and complex to manage.

F. PROTOTYPE MODEL



- Prototype is built for better understanding.
- Helps in clarifying requirements.
- Suitable when requirements are unclear.

\+ Better clarity and user involvement.
− Extra time for prototype.

3. COMPARISON AT A GLANCE

Model	Approach	Best Suited For	Flexibility	Risk Handling
Waterfall	Sequential	Small, Stable Projects	Low	Low
V-Model	Sequential (with Testing)	Safety / Quality Critical	Low	Medium
Iterative	Repeated Cycles	Projects with Partial Req.	High	Medium
Incremental	Step-by-Step Delivery	Medium to Large Projects	High	Medium
Spiral	Risk-Driven Cycles	Large, Complex Projects	High	High
Prototype	Prototype First	Unclear Requirements	High	Medium

KEY TAKEAWAY

Choose the right model based on project size, requirements clarity, budget, time and risk.

☆

@Aman_Views

★ COMPLETE MANUAL TESTING NOTES ★

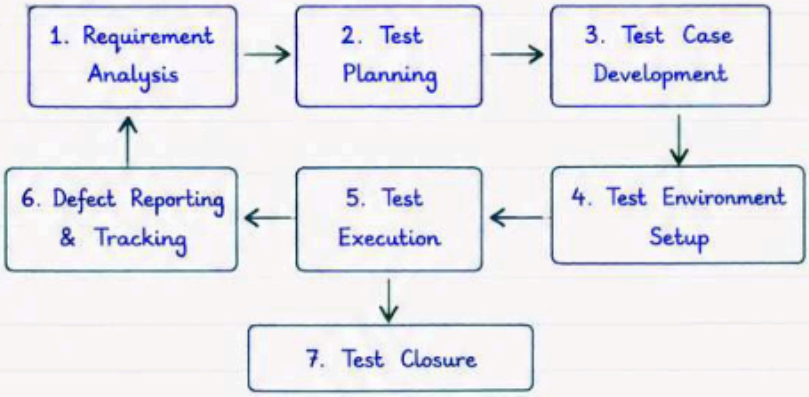
Everything from Beginner to Advanced.

PAGE 3 : STLC & TESTING PROCESS

1. WHAT IS STLC ?

- STLC (Software Testing Life Cycle) includes all activities performed during testing, from planning to closure.
- Helps ensure quality software delivery.

2. STLC (SOFTWARE TESTING LIFE CYCLE)



3. KEY STAGES (IN SHORT)

1. Requirement Analysis – Understand what to test.
2. Test Planning – Plan resources, strategy & schedule.
3. Test Case Development – Create test cases.
4. Test Environment Setup – Prepare environment.
5. Test Execution – Execute & verify test cases.
6. Defect Reporting & Tracking – Log & track defects.
7. Test Closure – Complete testing & sign-off.

4. ENTRY & EXIT CRITERIA

Entry Criteria (Start Testing)

- Requirements are clear.
- Test plan is approved.
- Test environment is ready.
- Test data is available.

Exit Criteria (End Testing)

- All test cases executed.
- No open critical defects.
- Test objectives are met.
- Sign-off from stakeholders.

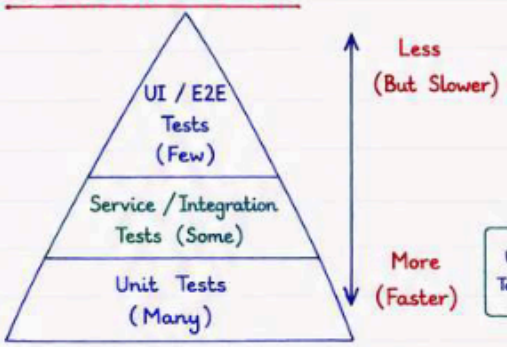
5. TEST DELIVERABLES

- Test Plan
- Test Cases
- Test Data
- Defect Report
- Test Summary Report
- Test Execution Report
- Requirement Traceability Matrix (RTM)

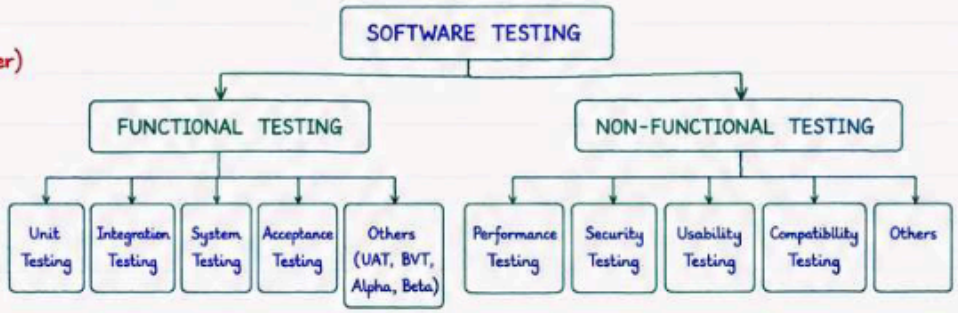
6. SDLC vs STLC

SDLC	STLC
Overall process	Testing process
Includes development & testing	Only testing
Bigger scope	Focus on quality

7. TESTING PYRAMID



8. COMPLETE TESTING CLASSIFICATION



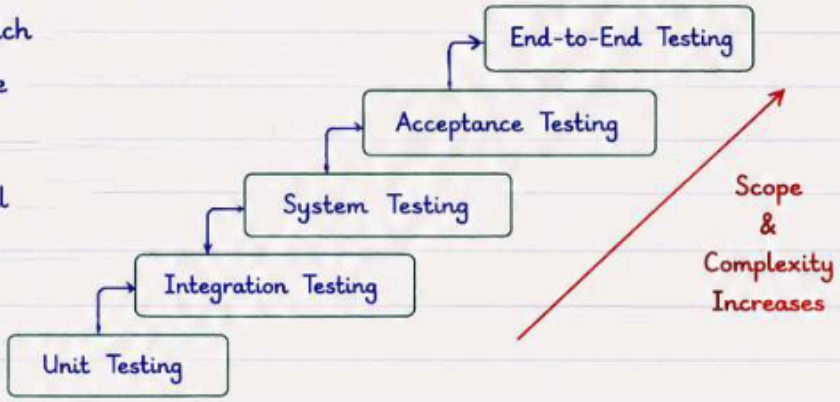
PAGE 4 : LEVELS OF SOFTWARE TESTING

1. WHAT ARE TESTING LEVELS ?

→ Testing levels define at which point of the application we test.

→ Each level has specific goal and responsibility.

2. TESTING LEVELS (SMALLEST TO LARGEST)



3. LEVELS EXPLAINED

Level	Purpose	Performed By
Unit Testing	Test individual units / components of the code.	Developers
Integration Testing	Test interaction between integrated modules.	Developers / QA
System Testing	Test the complete integrated system as a whole.	QA Testers
Acceptance Testing	Validate the system against business requirements.	Users / Customers
End-to-End Testing	Test complete business flow from start to finish.	QA Testers / Users

4. TYPES OF ACCEPTANCE TESTING

- Alpha Testing - Done at developer's site by internal users.
- Beta Testing - Done at customer's environment by real users.
- UAT (User Acceptance Testing) - Final validation by end users.

5. DEVELOPER vs QA RESPONSIBILITY

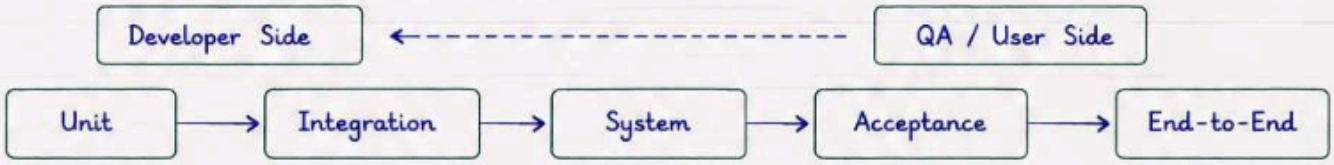
Developer

- Unit Testing
- Integration Testing
- Fixing Defects

QA Tester

- System Testing
- Acceptance Testing
- End-to-End Testing

6. LEVEL COMPARISON (QUICK VIEW)



Key Takeaway

Start testing small (unit) and move towards complete business flow (E2E) to ensure quality at every stage.

PAGE 5 : FUNCTIONAL TESTING

1. WHAT IS FUNCTIONAL TESTING ?

→ Functional testing verifies that the application works as per the specified requirements.

→ It focuses on features, inputs, outputs and business logic.

2. GOAL OF FUNCTIONAL TESTING

→ Validate each function of the application.

→ Ensure business requirements are met.

→ Verify input, output and data accuracy.

→ Ensure correct behaviour for all scenarios.

→ Deliver a bug-free user experience.

3. FUNCTIONAL TESTING WORKFLOW

Requirement Analysis → Test Case Design → Test Data Preparation → Test Case Execution → Defect Reporting → Retest & Closure

4. TYPES OF FUNCTIONAL TESTING

- Requirement Based Testing
- Feature / Functionality Testing
- Input Validation Testing
- Output Validation Testing
- Business Logic Testing
- User Interface Testing
- Database Testing
- API Functional Testing

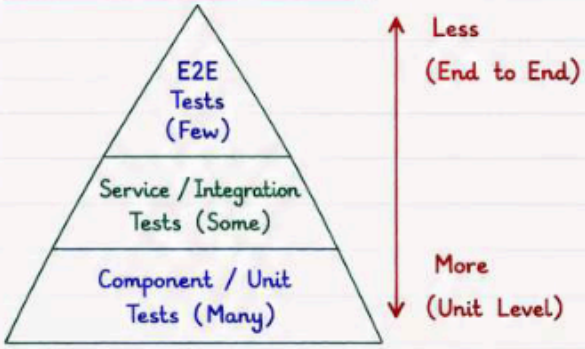
5. INPUT VALIDATION EXAMPLES

- Correct input → Should be accepted.
- Incorrect input → Should show error.
- Blank input → Should show error.
- Special characters → Should be handled.
- Boundary values → Should be validated.

6. POSITIVE vs NEGATIVE TESTING

Positive Testing	Negative Testing
• Test with valid inputs. • Verify expected functionality. • Example: Valid login with correct username and password.	• Test with invalid inputs. • Verify proper error handling. • Example: Login with wrong password.

7. TESTING PERSPECTIVE



8. EXAMPLE - LOGIN FUNCTION

Requirement	User should be able to login.
Input	Username, Password
Positive Test	Valid username & password → Login success
Negative Test	Invalid password → Error message
Output	Dashboard displayed after login
Data Validation	User record should exist in database

PAGE 6 : NON-FUNCTIONAL TESTING

1. WHAT IS NON-FUNCTIONAL TESTING ?

→ It checks how well the application works.

→ Focuses on performance, security, usability, reliability and more.

→ Ensures quality attributes of the software.

2. COMMON TYPES (NON-FUNCTIONAL)

Performance Testing	Checks speed & response time.
Load Testing	Checks under expected load.
Stress Testing	Checks beyond normal capacity.
Security Testing	Finds security vulnerabilities.
Usability Testing	Checks ease of use.
Compatibility Testing	Checks across devices, OS, browsers.
Reliability Testing	Checks stability over time.
Scalability Testing	Checks ability to handle growth.
Accessibility Testing	Checks for users with disabilities.

3. PERFORMANCE TESTING TYPES

Load Testing → Normal expected load

Stress Testing → Beyond normal capacity

Spike Testing → Sudden load increase

Volume Testing → Very large amount of data

4. FUNCTIONAL vs NON-FUNCTIONAL TESTING

Functional Testing	Non-Functional Testing
Checks WHAT the system does.	Checks HOW WELL the system works.
Validates features & functions.	Validates quality attributes.
Example: Login, Search, Add to Cart	Example: Speed, Security, Usability
Usually done by QA Testers.	Sometimes by specialized teams.
Ensures correctnecs of output.	Ensures performance & reliability.

5. QUALITY ATTRIBUTES (NON-FUNCTIONAL REQUIREMENTS)

Performance
Application should be fast and responsive.

Security
Protect data and prevent unauthorized access.

Usability
Application should be easy to learn and use.

Reliability
Application should work without failures.

Compatibility
Should work on different devices, OS and browsers.

Availability
Application should be available when needed.

Scalability
Should handle increased load and grow easily.

Maintainability
Should be easy to fix and update.

Portability
Should be easy to migrate to new environments.

Accessibility
Should be usable by people with disabilities.

6. KEY TAKEAWAY

Non-functional testing ensures the application is not only correct, but also fast, secure, reliable and user-friendly.

PAGE 7 : TESTING TECHNIQUES

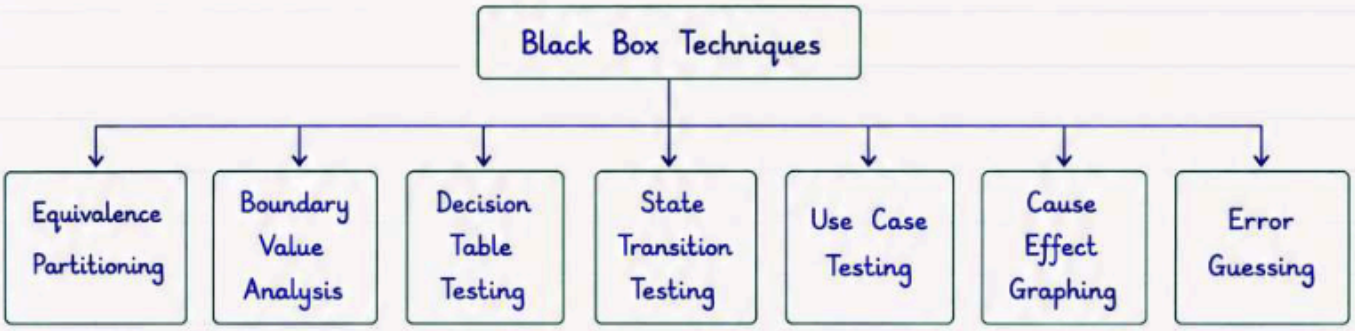
1. WHAT ARE TESTING TECHNIQUES ?

→ Testing techniques are methods used to design test cases and find defects effectively.

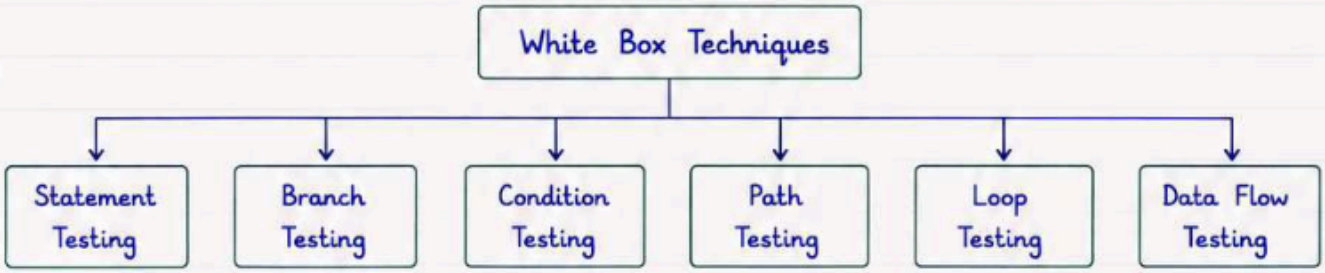
2. TYPES OF TESTING TECHNIQUES

- Black Box Testing Techniques (No knowledge of internal code)
- White Box Testing Techniques (Knowledge of internal code)

3. BLACK BOX TESTING TECHNIQUES



4. WHITE BOX TESTING TECHNIQUES



5. BLACK BOX vs WHITE BOX TESTING

Aspect	Black Box Testing	White Box Testing
Knowledge of Code	Not required	Required
Focus	Functionality	Internal logic / Code
Tester	QA Tester	Developer / QA
Techniques	EP, BVA, Decision Table, UAT etc.	Unit, Path, Loop, Branch etc.
Visibility	External (Input - Output)	Internal (Code structure)

6. KEY TAKEAWAY

→ Use Black Box to test WHAT the system does and White Box to test HOW the system works.

PAGE 8 : DEFECT MANAGEMENT

1. WHAT IS A DEFECT ?

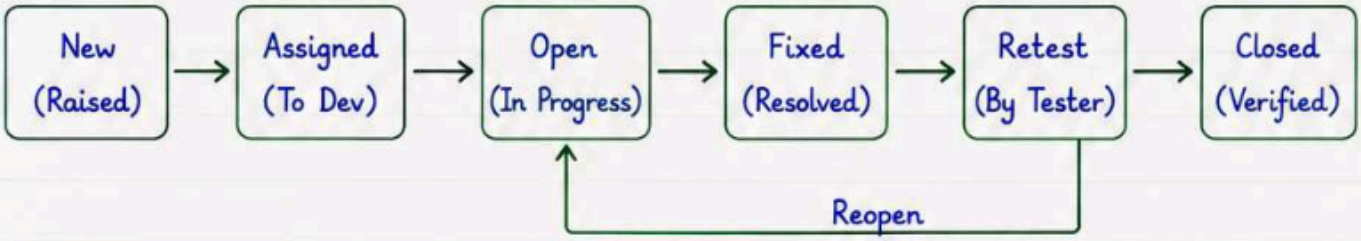
→ A defect is any deviation of the actual result from the expected result.

→ It represents a bug, error or issue in the software.

2. ERROR vs DEFECT

Error	Defect
• Mistake made by a human.	• Issue in the software.
• Cause of defect.	• Effect of error.
• Example: Wrong logic written by developer.	• Example: Login fails with valid credentials.

3. DEFECT LIFE CYCLE



4. DEFECT PRIORITY LEVELS

Priority	Description
P1 – Critical	System crash, data loss, major functionality not working.
P2 – High	Major functionality issue, workaround not available.
P3 – Medium	Minor functionality issue, workaround available.
P4 – Low	Cosmetic issue, enhancement, nice to have.

5. DEFECT SEVERITY LEVELS

- Blocker – System is unusable.
- Critical – Major feature is not working.
- Major – Core feature has serious issues.
- Minor – Minor issue, does not affect main flow.
- Trivial – Typo, alignment, UI issue.

PAGE 9 : BUG REPORTING

1. WHAT IS BUG REPORTING ?

→ Bug reporting is the process of documenting a defect in a system.

→ It helps developers to identify, reproduce and fix the issue.

2. PURPOSE OF BUG REPORTING

- Communicate defects clearly.
- Provide enough details to reproduce.
- Help in tracking and fixing.
- Improve product quality.
- Maintain project transparency.

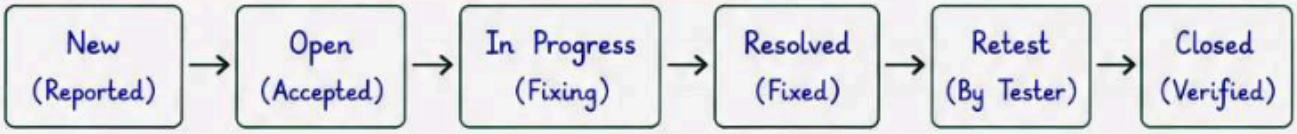
3. BUG REPORT TEMPLATE (KEY FIELDS)

Bug ID	Unique ID for the bug.
Title	Short and clear summary.
Reported By	Name of the tester.
Date	Date when bug was reported.
Module	Module / Feature name.
Environment	OS, Browser, Build/Version, Device, etc.
Priority	Low / Medium / High / Critical.
Severity	Trivial / Minor / Major / Critical.
Steps to Reproduce	Step by step method to reproduce the bug.
Expected Result	What should happen.
Actual Result	What actually happened.
Attachments	Screenshots, logs, files, videos, etc.
Status	New / Open / In Progress / Fixed / Closed / Rejected.

4. GOOD BUG REPORT CHARACTERISTICS

- Clear and concise.
- Reproducible.
- Complete with all details.
- No assumptions.
- One bug per report.
- Attach supporting evidence.

5. BUG LIFE CYCLE (IN BUG TRACKING TOOL)



PAGE 10 : TEST CASE DESIGN TECHNIQUES

1. WHAT IS TEST CASE DESIGN ?

→ Test case design is the process of creating test cases using different techniques to ensure good coverage.

→ Good test cases help to find defects effectively.

2. IMPORTANT POINTS

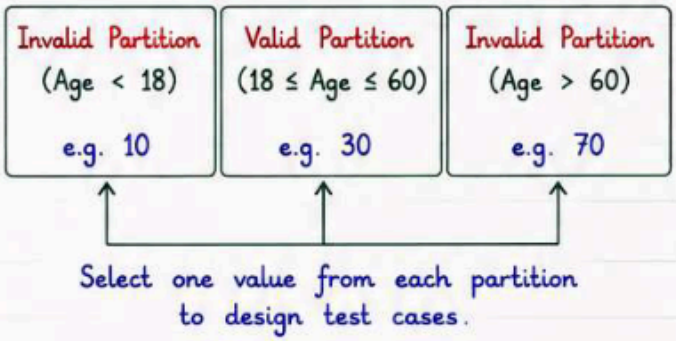
- Focus on requirements and functionality.
- Cover normal, boundary and error scenarios.
- Ensure reusability and maintainability.
- Avoid redundant test cases.
- Aim for maximum coverage with minimum test cases.

3. TEST CASE DESIGN TECHNIQUES

Technique	Description
Equivalence Partitioning	Divides input data into valid and invalid partitions. Test one value from each partition.
Boundary Value Analysis	Tests the values at the boundaries of input ranges.
Decision Table Testing	Used when business rules have multiple conditions and actions.
State Transition Testing	Validates behavior of system for different states and transitions.
Use Case Testing	Based on real user scenarios and workflows.
Error Guessing	Based on experience to guess where defects may occur.

4. EQUIVALENCE PARTITIONING EXAMPLE

Requirement: Age should be between 18 and 60.



5. BOUNDARY VALUE ANALYSIS EXAMPLE

Requirement: Age should be between 18 and 60.



Test boundary values to find defects at edges.

6. KEY TAKEAWAY

→ Use the right technique based on requirement.

→ Combine techniques for better coverage.

→ Well designed test cases = High quality software.

PAGE 11 : TEST CASE COMPONENTS

1. WHAT IS A TEST CASE ?

→ A test case is a set of conditions or variables, actions and expected results developed for a particular objective or test condition.

→ It is a document used to verify that the software works as expected.

2. IMPORTANCE

- Helps in systematic testing.
- Ensures good test coverage.
- Acts as a reference for testers.
- Helps in tracking progress.
- Improves reusability and maintenance.

3. TEST CASE COMPONENTS

Component	Description
Test Case ID	Unique identifier for the test case.
Test Case Title	Short description of the test case.
Objective	Purpose or goal of the test case.
Pre-conditions	Conditions that must be true before execution.
Test Data	Input values required for execution.
Test Steps	Step-by-step actions to be performed.
Expected Result	Result that should occur.
Actual Result	Result observed after execution.
Status	Pass / Fail / Blocked / Not Executed.
Remarks	Additional comments or notes.

4. TEST CASE TEMPLATE (EXAMPLE)

Test Case ID	:	TC_001
Test Case Title	:	Verify User Login with Valid Credentials
Objective	:	To verify that user can login successfully.
Pre-conditions	:	User is registered in the system.
Test Data	:	Username - testuser, Password - test@123
Test Steps	:	1. Open login page 2. Enter valid username and password 3. Click on Login button
Expected Result	:	User should be logged in and redirected to dashboard.
Actual Result	:	(To be filled during execution)
Status	:	(Pass / Fail)
Remarks	:	(If any)

PAGE 12 : TEST DATA

1. WHAT IS TEST DATA ?

→ Test data is the data used as input to execute test cases.

→ Good test data ensures accurate and reliable results.

2. TYPES OF TEST DATA

- Valid Data – Data that meets all business rules.
- Invalid Data – Data that violates one or more rules.
- Boundary Data – Data at the edges of valid range.
- Extreme Data – Data beyond the valid range.
- Normal Data – Common data used daily.

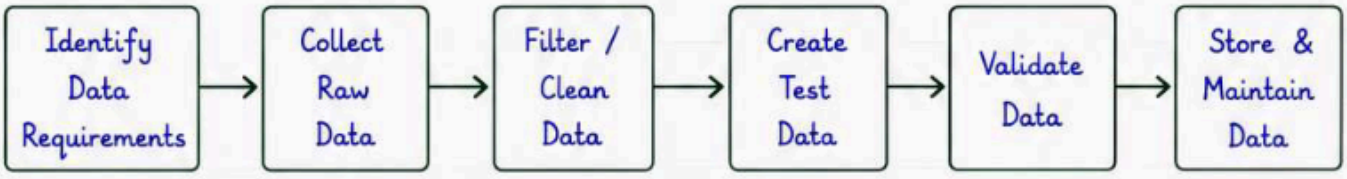
3. EXAMPLES OF TEST DATA

Type	Example (Age between 18 and 60)
Valid Data	25, 30, 45
Invalid Data	abc, -5, 70
Boundary Data	18, 60
Extreme Data	17, 61, -100, 999
Normal Data	28, 32, 40

4. GOOD TEST DATA CHARACTERISTICS

- Accurate and up-to-date.
- Covers all scenarios.
- Easy to maintain and reusable.
- Non-duplicate and consistent.

5. TEST DATA PREPARATION PROCESS



PAGE 13 : TEST ENVIRONMENT

1. WHAT IS TEST ENVIRONMENT ?

→ Test environment is the setup of hardware, software, network and data required to test the application.

→ It should be similar to production environment to get accurate results.

2. IMPORTANCE

- Ensures accurate and reliable testing.
- Helps to identify defects early.
- Prevents environment related issues.
- Provides real-life like conditions.
- Improves product quality.

3. COMPONENTS OF TEST ENVIRONMENT

Component	Description
Hardware	Servers, computers, mobile devices, storage, etc.
Software	OS, databases, browsers, tools, application under test, etc.
Network	LAN, WAN, internet connectivity, firewalls, etc.
Data	Test data, databases, files required for testing.
Tools	Testing tools, defect tracking tools, performance tools, etc.
People	Testers, developers, DBAs, system admins, etc.
Documentation	Test cases, test data documents, environment setup guide.

4. TYPES OF ENVIRONMENTS

Environment	Purpose
Development Environment	Used by developers to build and unit test the application.
QA / Test Environment	Used by testers to perform functional and non-functional testing.
Staging Environment	Pre-production environment used for final validation.
Production Environment	Live environment where actual users access the application.

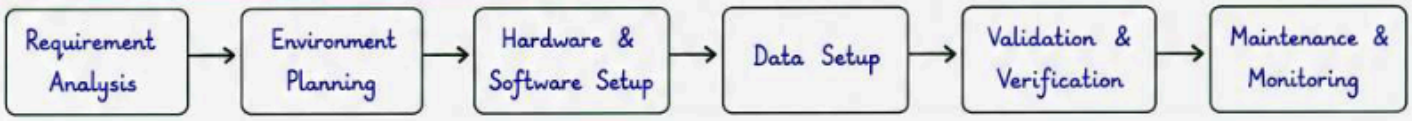
5. GOOD TEST ENVIRONMENT CHARACTERISTICS

- Similar to production environment.
- Stable and available when needed.
- Isolated (changes in one environment should not affect others).
- Easy to setup, maintain and support.
- Secure and backed up regularly.
- Has required tools, data and documentation.

NOTE

An unstable or incorrect environment can lead to wrong results and waste of time and effort.

6. TEST ENVIRONMENT SETUP PROCESS



PAGE 14 : TEST EXECUTION

1. WHAT IS TEST EXECUTION ?

→ Test execution is the process of running test cases to verify that the actual results match the expected results.

→ It is performed after test planning, test case design and environment setup.

→ It helps to identify defects and validate the application.

2. OBJECTIVES OF TEST EXECUTION

- Execute test cases as per test plan.
- Identify defects and report them.
- Verify that defects are fixed.
- Ensure application meets requirements.
- Provide feedback on product quality.
- Maintain execution records and results.

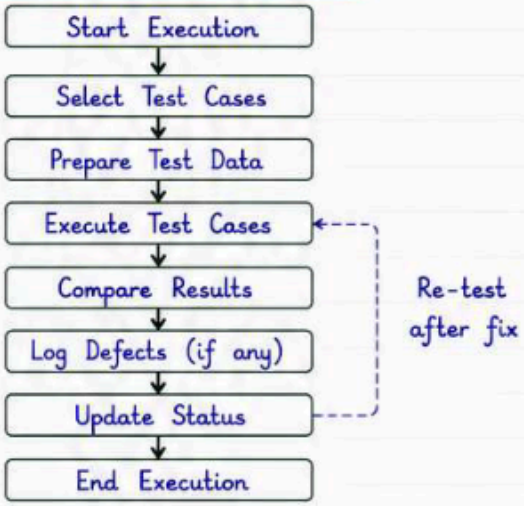
3. TEST EXECUTION TASKS

Task	Description	Responsibility
Review Test Cases	Review test cases to ensure they are clear and executable.	Tester
Prepare Test Data	Prepare necessary test data as per test case requirements.	Tester
Execute Test Cases	Run test cases and compare actual results with expected results.	Tester
Log Defects	Log defects with proper details and assign to developer.	Tester
Re-test Defects	Verify that the defect is fixed.	Tester
Update Test Status	Update test case status and maintain execution records.	Tester

4. TEST CASE STATUS

Status	Description
Not Executed	Test case not yet executed.
Pass	Test case executed successfully.
Fail	Test case failed.
Blocked	Test case cannot be executed due to some blocker.
Skipped	Test case skipped as per decision.
Retest	Test case executed again after fix.

5. TEST EXECUTION FLOW



6. GOOD TEST EXECUTION PRACTICES

- Understand the test cases before execution.
- Use proper test data.
- Follow test steps as written.
- Log defects with clear and complete details.
- Re-test after defect fix.
- Update test status accurately.
- Maintain execution log for tracking.
- Communicate issues immediately.

PAGE 15 : DEFECT MANAGEMENT

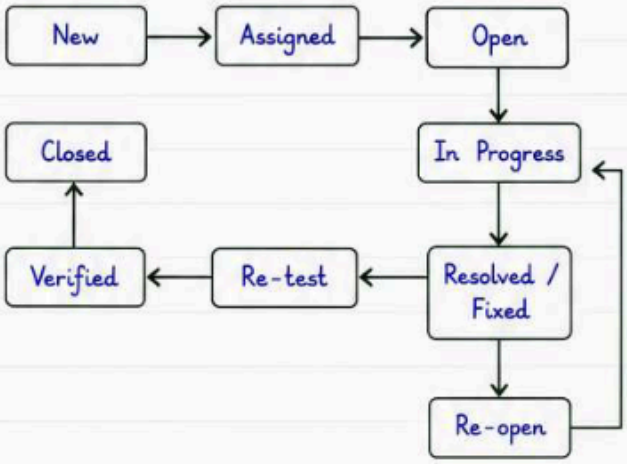
1. WHAT IS A DEFECT ?

→ A defect (or bug) is any deviation from the expected result.

→ Defects are found during test execution and need to be logged, tracked and resolved.

→ Proper defect management ensures quality and helps prevent similar issues in future.

2. DEFECT LIFE CYCLE



3. DEFECT STATUS EXPLANATION

Status	Description
New	Defect is logged and not yet assigned.
Assigned	Defect is assigned to a developer.
Open	Developer starts working on the defect.
In Progress	Developer is investigating or fixing the defect.
Resolved / Fixed	Developer has fixed the defect.
Re-test	Tester verifies the fix.
Verified	Tester confirms the defect is fixed.
Re-open	Defect is not fixed or issue still exists.
Closed	Defect is fixed and verified. No further action required.

4. DEFECT PRIORITY

Priority	Description
P0 - Critical	System crash, data loss, major functionality broken.
P1 - High	Major functionality not working as expected.
P2 - Medium	Minor functionality issues, work-around available.
P3 - Low	Cosmetic issues, enhancement requests.

5. DEFECT SEVERITY

Severity	Description
Blocker	Application is completely unusable.
Critical	Major features not working, but app partially usable.
Major	Some features affected, no work-around.
Minor	Minor issues, workaround available.
Trivial	Cosmetic issues, no impact on functionality.

6. GOOD DEFECT REPORT CHARACTERISTICS

- Clear and concise title.
- Steps to reproduce.
- Actual result vs Expected result.
- Screenshots / logs / videos if possible.
- Environment details (OS, Browser, Build etc.).
- Priority and severity assigned.
- No duplicate defects.
- Easy to understand for developer.

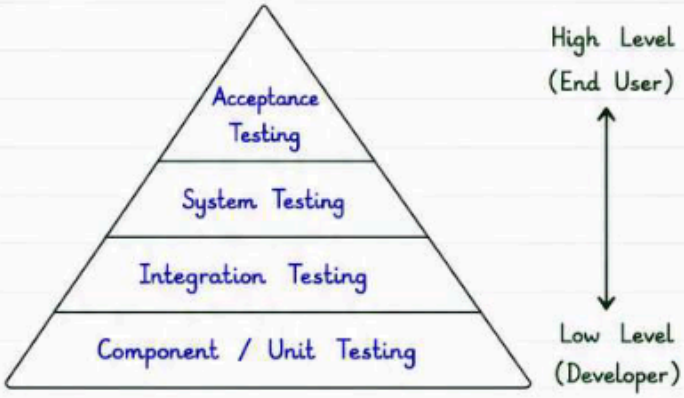
PAGE 16 : LEVELS OF TESTING

1. WHAT IS LEVELS OF TESTING ?

→ Levels of testing are performed in a sequence to ensure the product is thoroughly tested from small components to the entire system.

→ Each level has a specific objective and is done by different teams.

2. TESTING LEVELS (OVERVIEW)



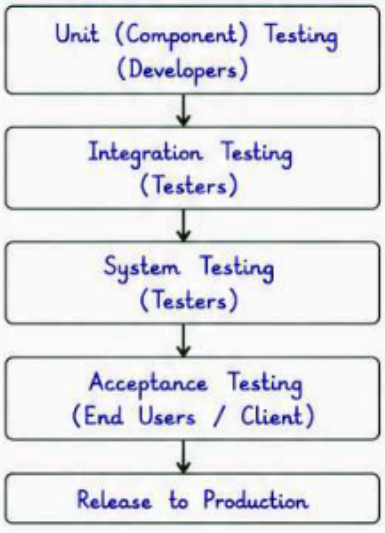
3. LEVELS OF TESTING - DETAILS

Level	Performed By	Objective	Test Basis	Example
1. Unit (Component) Testing	Developers	To test individual units / components of the application.	Requirement Design	Test a function, class, method separately.
2. Integration Testing	Testers	To test integrated modules and interface between modules.	Design Interface Specs	Test login module with database module.
3. System Testing	Testers	To test the complete system as a whole against requirements.	Requirements Specification	Test the entire application end to end.
4. Acceptance Testing	End Users / Client	To validate the system meets business needs and is ready to use.	Business Requirements User Scenarios	UAT by client before production release.

4. LEVELS COMPARISON

Aspect	Unit Testing	Integration Testing	System Testing	Acceptance Testing
Focus	Individual Component	Module Interaction	Complete System	Business Validation
Performed By	Developers	Testers	Testers	End Users
Test Basis	Design / Code	Interface Specs	SRS / FRD	BRD / User Scenarios
Environment	Developer Environment	Integration Environment	Test Environment	UAT Environment
Test Data	Small & Simple	Module Specific	End to End Data	Realistic Data
Purpose	Verify Code Logic	Verify Module Interface	Verify System Functionality	Verify Business Readiness

5. TESTING FLOW ACROSS LEVELS



6. KEY POINTS

- Testing starts from small components and moves to the complete system.
- Defects found in early levels are cheaper to fix.
- Each level has different objectives and responsibilities.
- Higher level testing depends on the success of lower level testing.
- Proper levels of testing ensure quality and customer satisfaction.

PAGE 17 : TEST DATA & ENVIRONMENTS

1. WHAT IS TEST DATA ?

→ Test data is the data used to verify the functionality of an application.

→ It helps in checking whether the system works as expected with different inputs, conditions and scenarios.

→ Good test data ensures better coverage, more accurate results and higher quality software.

2. TYPES OF TEST DATA

Type	Description
Valid Data	Correct and expected data used for positive testing.
Invalid Data	Incorrect or wrong data used for negative testing.
Boundary Data	Data at the edges (min/max) of allowed values.
Highly Confidential	Sensitive data like passwords, credit card details, etc.
Database Data	Data directly from the database (real or dummy).

3. SOURCES OF TEST DATA

- Production data (anonymized)
- Database queries
- Third-party tools / data generators
- Manual creation (spreadsheets, CSV, etc.)
- Synthetic data (auto-generated)

4. BEST PRACTICES FOR TEST DATA

- Use realistic and relevant data.
- Include positive, negative and boundary data.
- Keep data up to date and well organized.
- Avoid using real customer data (privacy).
- Use data masking for sensitive information.

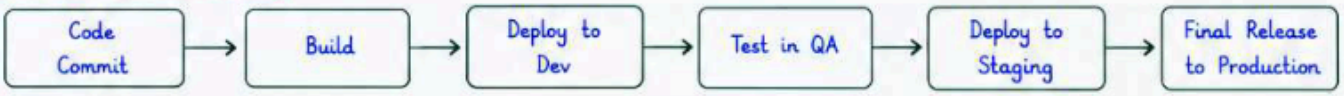
5. TEST ENVIRONMENT VS TEST DATA

Test Environment	Test Data
1. It is the setup where tests are executed.	It is the data used in the tests.
2. Includes hardware, software, network, OS, etc.	Includes valid, invalid, boundary, etc. data.
3. Example: Dev, QA, Staging, Production.	Example: User data, product data, transaction data.
4. Ensures the system is ready for testing.	Ensures the application behaves correctly with different inputs.

6. TYPES OF ENVIRONMENTS

Environment	Purpose	Example
Development (Dev)	Coding and initial testing.	Local machine / Dev server
QA / Test	Functional and non-functional testing.	QA server
Staging	Pre-production testing (almost production).	Staging server
Production	Live for real users.	Live server

7. DEPLOYMENT FLOW



8. KEY POINTS

- Test data and environment are different but both are essential.
- Use separate environments for different stages.
- Keep test data safe and secure.
- Environment should be stable and reliable.
- Fresh data gives better results.
- Proper data and environment reduce defects and save time/cost.